МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине

«Операционные системы»

«Введение в Xv6»

Работу выполнил:
Студенты группы P3411
Болорболд Аригуун

Лектор:
Клименков Сергей Викторович
Практик:
Смирнов Виктор Игоревич

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
   1.1. Часть 1. Pingpong 3
   1.2. Часть 2. Dump 3
   1.3. Часть 3*. Dump2 4
   1.4. Часть 4*. Дополнительное задание 5
2. Ход работы 6
   2.1. Часть 1. Реализация `pingpong.c` 6
   2.2. Часть 2 и 3. Дамп-тесты 8
   2.3. Часть 4. Символические ссылки 14
3. Вывод 19

1. Текст задания

1.1. Часть 1. Pingpong

Прежде чем перейти к основной части курса, познакомимся подробнее с `Xv6` и её системными вызовами. Мы попробуем написать немного user-space кода. Научитесь обмениваться данными между процессами с помощью специальных FIFO-каналов — Unix pipes.

Реализуйте программу `user/pingpong.c`, которая должна:

1. Создать пайп.
2. Создать дочерний процесс.
3. Отправить сообщение `ping` из родительского процесса в дочерний.
4. Прочитать их в дочернем процессе, вывести `<child pid>: got <message>` и отправить сообщение `pong` в ответ.
5. Прочитать ответ в родительском процессе, вывести `<parent pid>: got <message>`.
6. Для проверки корректности реализации найдите в исходном коде максимальный размер данных, который может помещаться в pipe без считывания, измените его на 1 байт и проверьте работоспособность своего решения. После этого верните pipe его изначальный размер.

Советы для выполнения задания:

- Вам понадобится несколько системных вызовов — `pipe`, `fork`, `read`, `write`, `getpid`. Воспользуйтесь утилитой `man`, чтобы узнать, что делают эти вызовы и как ими пользоваться — поведение в `Xv6` не особо отличается от других Unix-подобных операционных систем.
- Вместо привычных вам `stdlib.h` и `stdio.h` доступна местная стандартная библиотека — `user/ulib.c`, а также `/user/printf.c` и `user/umalloc.c`. Посмотрите на другие программы в директории `/user`, чтобы понять, как ей пользоваться.
- Добавьте программу в `UPROGS` в `Makefile`, чтобы она скомпилировалась.
- В программах для `Xv6` обязательно нужно вызывать `exit(0)` для выхода.

1.2. Часть 2. Dump

В прошлой части ЛР мы использовали системные вызовы, например, `pipe` и `fork`. Задача системных вызовов — дать программам из user-space возможность выполнять привилегированные команды. Реализуем новый системный вызов `dump`. Он будет выводить на экран состояние регистров `s2 –s11` вызывающего процесса. Чтобы системный вызов был доступен из user-space, добавим в файл `user/user.h` объявление функции `dump`, как это сделано для других системных вызовов. В файл `user/usys.pl` добавьте

строку `entry("dump")` — он отвечает за генерацию ассемблерных инструкций для совершения системного вызова.

Теперь реализуем сам системный вызов. Для этого добавьте функцию `dump` в файл `kernel/proc.c`. Текущий процесс можно получить с помощью функции `myproc`. Структура `proc` содержит поле `trapframe`, в котором и находятся значения всех регистров. Избегайте лишней копипасты при выводе регистров. Все регистры в `Xv6` 64-битные, однако в рамках данного задания для каждого регистра вам нужно вывести лишь младшую 32-битную часть.

Наконец, отредактируйте файлы `kernel/syscall.h`, `kernel/sysproc.c` и `kernel/syscall.c` так, чтобы появилась возможность вызвать `dump` из user-space. Посмотрите, как реализованы другие вызовы, и сделайте аналогично. Осталось собрать `Xv6`. Запустите утилиту `user/dumptests.c` и сравните фактические значения регистров и результат вашего системного вызова.

- Функцию `dump` нужно также определить заголовочном файле в `kernel/defs.h`.
- Системный вызов должен возвращать 0 при успешном завершении, и код ошибки в остальных случаях. Наш системный вызов всегда завершается успешно.
- Если при запуске `dumptests` выводится сообщение о том, что системный вызов `dump` не найден, то попробуйте пересобрать `Xv6` с нуля.

1.3. Часть 3*. Dump2

Мы бы могли использовать системный вызов `dump`, чтобы написать собственный отладчик. Однако, у него есть два недостатка. Во-первых, он выводит значение регистров на экран, и мы не можем обработать эти значения в user-space. Во-вторых, он позволяет узнать значения регистров только у текущего процесса, что делает невозможным отладку другого процесса. Напишем ещё один системный вызов, чтобы исправить эти недостатки — `dump2`.

У этого вызова будет три аргумента:

1. `int pid` — номер процесса, для которого запрашивается значение регистра
2. `int register_num` — номер регистра, число от 2 до 11
3. `uint64 *return_value` — адрес, по которому необходимо вернуть значение

Обратите внимание, что в целях безопасности регистры процесса может смотреть только сам процесс и его предки.

В этом системном вызове, в отличие от `dump`, вам понадобится корректно обрабатывать и возвращать ошибки:

- Верните `-1`, если вызвавший процесс не имеет прав смотреть требуемый регистр
- Верните `-2`, если процесса с таким идентификатором не существует
- Верните `-3`, если передан некорректный номер регистра
- Верните `-4`, если не удалось записать данные по переданному адресу

1.4. Часть 4*. Дополнительное задание

Реализовать символические ссылки. Показывать исполнение, отличие от жесткой ссылки и обработку циклического вызова ссылок.

2. Ход работы

2.1. Часть 1. Реализация pingpong.c

```
int
main(void)
{
    int pipe_desc[2];
    char buf[6];

    int pipe_code = pipe(pipe_desc);

    if (pipe_code < 0)
    {
        fprintf(STDERR, "pipe_err");
        exit(FAIL);
    }

    pid_t pid = fork();

    if (pid < 0) {
        fprintf(STDERR, "fork_err\n");
        exit(FAIL);
    }

    const int len_child_msg = sizeof(FROM_CHILD_MSG);
    const int len_parent_msg = sizeof(FROM_PARENT_MSG);

    if (pid > 0)
    {
        // Родительский процесс
        int bytes_written = custom_write(pipe_desc[1],
FROM_CHILD_MSG, len_child_msg);
        if (bytes_written < 0)
        {
            fprintf(STDERR, "error(pid %d): write failed in parent.
\n", getpid());
            exit(FAIL);
        }
        if (bytes_written != len_child_msg) {
            fprintf(STDERR, "warning(pid %d): parent wrote %d bytes
(expected: %d).\n",
                getpid(), bytes_written, len_child_msg);
        }
        wait(0); // Остановка всех родительских процессов до
завершения всех дочерних процессов
```

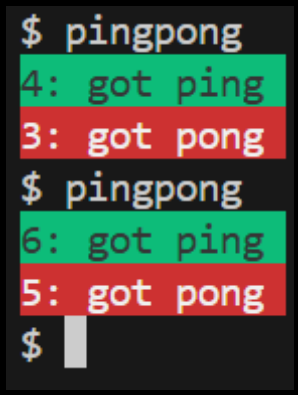
```
        int bytes_read = custom_read(pipe_desc[0], buf,
len_parent_msg);
        if (bytes_read < 0)
        {
            fprintf(STDERR, "error(pid %d): read failed in parent.
\n", getpid());
            exit(FAIL);
        }
        if (bytes_read < len_parent_msg)
        {
            fprintf(STDERR, "warning(pid %d): parent expected %d
bytes but got %d. Possible EOF.\n",
                getpid(), len_parent_msg, bytes_read);
        }
        fprintf(STDOUT, "%s%d: got %s %s\n", REDB, getpid(), buf,
CRESET);
    }
    else
    {
        // Дочерний процесс
        int bytes_read = custom_read(pipe_desc[0], buf,
len_child_msg);
        if (bytes_read < 0) {
            fprintf(STDERR, "error(pid %d): read failed in child.
\n", getpid());
            exit(FAIL);
        }
        if (bytes_read < len_child_msg) {
            fprintf(STDERR, "error(pid %d): child expected %d bytes
but got %d (EOF?).\n",
                    getpid(), len_child_msg, bytes_read);
            exit(FAIL);
        }

        fprintf(STDOUT, "%s%d: got %s %s\n", GRNB, getpid(), buf,
CRESET);

        int written = custom_write(pipe_desc[1], FROM_PARENT_MSG,
len_parent_msg);
        if (written < 0) {
            fprintf(STDERR, "error(pid %d): write failed in child.
\n", getpid());
            exit(FAIL);
        }
        if (written != len_parent_msg) {
            fprintf(STDERR, "warning(pid %d): child wrote %d bytes
```

```
(expected %d).\n",
                    getpid(), written, len_parent_msg);
        }
    }
    exit(NORMAL);
}
```

После добавления программы в `Makefile` оболочки:

```
$U/_pingpong\
```

провел его тестирование:



Картина показывает ожидаемое поведение.

2.2. Часть 2 и 3. Дамп-тесты

```
uint64
dump(void)
{
  struct proc* current_process = myproc();
  uint64 *registers = &current_process->trapframe->s2;
  for(int i = 0; i < 10; ++i)
  {
    printf("s%d = %d\n", i + 2, (uint32) registers[i]);
  }
  return 0;
}

uint64
dump2(int pid, int register_num, uint64* return_value)
{
  enum error_code {
    OK = 0,
    NO_RIGHTS = -1,
    INVALID_PID = -2,
    INVALID_REGISTER = -3,
    FAILED_WRITE = -4
```

```
  };

  if(register_num > 11 || register_num < 2)
  {
    return INVALID_REGISTER;
  }

  struct proc* current_proc = myproc();
  struct proc* seeked_proc;

  uint64* register_state_ptr;
  ushort pid_found = 0;

  for(seeked_proc = proc; seeked_proc < &proc[NPROC]; seeked_proc+
+)
  {
    acquire(&seeked_proc->lock);
    if(seeked_proc->pid == pid)
    {
      pid_found = 1;
      break;
    } else {
      release(&seeked_proc->lock);
    }
  }

  if(!pid_found)
  {
    return INVALID_PID;
  }

  acquire(&wait_lock);
  if(current_proc->pid != seeked_proc->pid && current_proc->pid !=
seeked_proc->parent->pid)
  {
    release(&wait_lock);
    release(&seeked_proc->lock);
    return NO_RIGHTS;
  }

  register_state_ptr = &(seeked_proc->trapframe->s2) - 2 +
register_num;
  if(copyout(current_proc->pagetable, (uint64) return_value, (char
*) register_state_ptr, sizeof(uint64)) < 0)
  {
    release(&wait_lock);
```

```
    release(&seeked_proc->lock);
    return FAILED_WRITE;
  }

  release(&wait_lock);
  release(&seeked_proc->lock);
  return OK;
}
```

И после настройки (здесь это обязательно, так как они являются системными вызовами) в ядре:

```
// defs.h
// ...
uint64          dump(void);
uint64          dump2(int, int, uint64*);
// ...

// sysproc.c
// ...
uint64
sys_dump(void)
{
  return dump();
}


uint64
sys_dump2(void)
{
  int pid, register_num;
  uint64 return_value;

  argint(0, &pid);
  argint(1, &register_num);
  argaddr(2, &return_value);

  return dump2(pid, register_num, (uint64 *) return_value);
}
// ...

// syscall.h
// ...
#define SYS_dump   22
#define SYS_dump2  23
// ...
```

```
// syscall.c
// ...
extern uint64 sys_dump(void);
extern uint64 sys_dump2(void);
// ...
[SYS_dump]    sys_dump,
[SYS_dump2]   sys_dump2
```

И в пользовательском пространстве:

```
// user.h
...
uint64 dump();
uint64 dump2(int pid, int register_num, uint64* return_value);
```

```
# usys.h
# ...
entry("dump");
entry("dump2");
```

Провел тестирование: dumptests:

```
$ dumptests
dump tests started
dump syscall found. Start testing
#####################
#                   #
#   initial state   #
#                   #
#####################
s2 = 99
s3 = 32
s4 = 84215045
s5 = 84215045
s6 = 84215045
s7 = 84215045
s8 = 84215045
s9 = 84215045
s10 = 84215045
s11 = 84215045
```

```
######################
#                    #
#       test 1       #
#                    #
######################
#                    #
#  expected values   #
#                    #
######################
# s2  = 2            #
# s3  = 3            #
# s4  = 4            #
# s5  = 5            #
# s6  = 6            #
# s7  = 7            #
# s8  = 8            #
# s9  = 9            #
# s10 = 10           #
# s11 = 11           #
######################
s2 = 2
s3 = 3
s4 = 4
s5 = 5
s6 = 6
s7 = 7
s8 = 8
s9 = 9
s10 = 10
s11 = 11
```

```
######################
#                    #
#      test 2        #
#                    #
######################
#                    #
#  expected values   #
#                    #
######################
# s2 = 1             #
# s3 = -12           #
# s4 = 123           #
# s5 = -1234         #
# s6 = 12345         #
# s7 = -123456       #
# s8 = 1234567       #
# s9 = -12345678     #
# s10 = 123456789    #
# s11 = -1234567890  #
######################
s2 = 1
s3 = -12
s4 = 123
s5 = -1234
s6 = 12345
s7 = -123456
s8 = 1234567
s9 = -12345678
s10 = 123456789
s11 = -1234567890
```

```
######################
#                    #
#      test 3        #
#                    #
######################
#                    #
#  expected values   #
#                    #
######################
# s2 = 2147483647    #
# s3 = -2147483648   #
# s4 = 1337          #
# s5 = 2020          #
# s6 = 3234          #
# s7 = 3235          #
# s8 = 3236          #
# s9 = 3237          #
# s10 = 3238         #
# s11 = 3239         #
######################
s2 = 2147483647
s3 = -2147483648
s4 = 1337
s5 = 2020
s6 = 3234
s7 = 3235
s8 = 3236
s9 = 3237
s10 = 3238
s11 = 3239
4 tests were ran
```

dump2tests:

```
$ dump2tests
dump2 tests started
dump2 syscall found. Start testing
test 1 started
[SUCCESS] test 1 passed
test 1 finished
test 2 started
[SUCCESS] test 2 passed
test 2 finished
test 3 started
[SUCCESS] test 3 passed
test 3 finished
test 4 started
[INFO] testing nonexisting proccess
[OK] nonexisting proccess
[INFO] testing illegal access to registers
[OK] illegal access to registers
[INFO] testing incorrect number of register
[OK] incorrect number of register
[INFO] testing invalid memory address
[OK] invalid memory address
[SUCCESS] test 4 passed
test 4 finished
4 tests were run. 4 tests passed
```

2.3. Часть 4. Символические ссылки

Чтобы выполнить эту задачу, мне пришлось создать новый системный вызов в `sysfile.c`, так как работа проходит исключительно с файловой системой.

```
uint64
sys_symlink(void) {
  char target[MAXPATH], path[MAXARG];
  if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) <
0)
  {
    return -1;
  }
```

```
  begin_op();
  struct inode *ip = create(path, T_SYMLINK, 0, 0);
  if(ip == 0)
  {
    end_op();
    return -1;
  }
  int len = strlen(target);
  writei(ip, 0, (uint64)&len, 0, sizeof(int));
  writei(ip, 0, (uint64)target, sizeof(int), len + 1);
  iupdate(ip);
  iunlockput(ip);
  end_op();
  return 0;
}
```

Также приходилось добавить новые переменные и модифицировать существующие файлы для работы с новым видом объекта файловой системы:

```
// fcntl.h
// ...
#define O_NOFOLLOW 0x100

// stat.h
// ...
#define T_SYMLINK 4 // Symbolic link

// sysfile.c
static struct inode*
create(char *path, short type, short major, short minor) {
  // ...
  if(type == T_SYMLINK)
      return ip;
  // ...
}
// ...
uint64
sys_open(void)
{
  // ...
  if((ip->type == T_SYMLINK) && !(omode & O_NOFOLLOW)){
    int count = 0;
    while (ip->type == T_SYMLINK && count < 10){
      int len = 0;
      readi(ip, 0, (uint64)&len, 0, sizeof(int));
      if (len > MAXPATH){
```

```
        panic("open: corrupted symlink inode");
      }
      readi(ip, 0, (uint64)path, sizeof(int), len + 1);
      iunlockput(ip);
      if((ip = namei(path)) == 0){
        end_op();
        return -1;
      }
      ilock(ip);
      count++;
    }
    if(count >= 10){
      iunlockput(ip);
      end_op();
      return -1;
    }
  }
  // ...
}
```

Потом, после настройки в ядре:

```
// defs.h
// ...
uint64          symlink(void);
// ...

// syscall.h
// ...
#define SYS_symlink 24
// ...

// syscall.c
// ...
extern uint64 sys_symlink(void);
// ...
[SYS_symlink] sys_symlink
// ...
```

И изменений `ln.c` (потому что символические ссылки создаются через `ln -s` в типичных линуксовых дистрибутивах):

```
int
main(int argc, char *argv[])
{
  int is_hardlink = 1;
  if(argc >= 2 && (argv[1][0] == '-' && argv[1][1] == 's' &&
```

```
argv[1][2] == '\0'))
  {
    is_hardlink = 0;
    argc--;
    argv++;
  }

  if(argc != 3)
  {
    fprintf(2, "Usage: ln old new\n");
    exit(1);
  }

  if(is_hardlink)
  {
    if(link(argv[1], argv[2]) < 0)
    {
      fprintf(2, "link %s %s: failed\n", argv[1], argv[2]);
      exit(1);
    }
  }
  else
  {
    if(symlink(argv[1], argv[2]) < 0)
    {
      fprintf(2, "link (-s) %s %s: failed\n", argv[1], argv[2]);
      exit(1);
    }
  }
  exit(0);
}

// user.h
// ...
int symlink(char*, char*);
// ...

// ...
entry("symlink");
```

Провел тестирование.

1. Создание ссылки:

```
$ echo 'aint no party like a Diddy party' > original.txt
$ ln original.txt hard.txt
$ ln -s original.txt symlink.txt
```

```
alloctest      2 22 37752
cowtest        2 23 40928
lazytests      2 24 39552
console        3 25 0
original.txt   2 26 35
hard.txt       2 26 35
symlink.txt    2 26 35
```

```
$ cat hard.txt
'aint no party like a Diddy party'
$ cat original.txt
'aint no party like a Diddy party'
$ cat symlink.txt
'aint no party like a Diddy party'
```

Символическая ссылка создалась так же, как и жесткая.

2. Отличие от жесткой ссылки

```
$ rm original.txt
$ cat hard.txt
'aint no party like a Diddy party'
$ cat symlink.txt
cat: cannot open symlink.txt
$
```

В отличие от жесткой ссылки, символическая ссылка не открывается. Это связано с тем, что символическая ссылка указывает на название файла, а жесткая — на саму `inode` файла. `inode` не удаляется с ОС, пока не удаляются все файлы с таким же значением `inode`, поэтому жесткая ссылка работает даже когда удалился оригинальный файл. В отличие от этого, символическая ссылка не открывается, потому что указываемый оригинальный файл удалился, и без него она не будет работать, что соответствует требованию.

3. Циклический вызов

```
$ echo 'aaaaa' > cycle1.txt
$ ln -s cycle1.txt cycle2.txt
$ ln -s cycle2.txt cycle1.txt
```

ОС должна корректно обрабатывать случай, когда 2 символических ссылок имеют ссылку на друг друга, или когда ссылки создают циклическую зависимость.

```
$ cat cycle2.txt
cat: cannot open cycle2.txt
$ cat cycle1.txt
cat: cannot open cycle1.txt
```

Что подтверждается этой картинкой. Конечно, можно было это реализовать красивее (сообщение, которое появляется только в этом случае), но это более чем устраивает.

3. Вывод

В ходе выполнения работы были освоены принципы работы с пайпами для передачи процесса, с написанием системных вызовов и с файловой системы (частично) путём реализации символических ссылок. Эта работа дает хорошую базу при работе со следующими, более сложными работами, как аллокаторов и файловой системы.